Karadeniz Teknik Üniversitesi

Fen Fakültesi – Bilgisayar Bilimleri Bölümü

OBYS Client/Server Mimarisi

Bölüm 13 – Hafta 13

Yazılım Geliştirme II (BILB2000)

Bahar Dönemi – 13. Hafta

Bu Haftanın Planı

1. Ünite 1: OBYS Sunucu Tasarımı
2. Ünite 2: OBYS İstemci Tasarımı
3. Ünite 3: Veri Senkronizasyonu
4. Ünite 4: Güvenlik Temelleri

Hafta 12'den Hafta 13'e Köprü

Geçen hafta nereye geldik?

Hafta 12'de **ağ temellerini** kurduk: OSI/TCP-IP modeli, POSIX soketler, `QTcpSocket`/`QTcpServer`, length-prefix çerçeveleme ve JSON tabanlı OBYS Protokol v1 *taslağı*. Sonunda küçük bir echo sunucusu çalışıyordu.

Bu hafta dolduracağımız iskelet

- Çok istemcili gerçek OBYS sunucusu
- Asenkron OBYS istemcisi (GUI)
- Eş zamanlı veri erişimi (shared_mutex)
- Audit log + checkpoint
- Parola hash'i, TLS, yetkilendirme

Birleşim noktası

- STL Depo (Bölüm 1-3)
- Qt GUI (Bölüm 4-8)
- Thread-safe yapılar (Bölüm 9-11)
- Ağ + protokol (Bölüm 12)
- = **tam dağıtık OBYS**

Ünite 1: OBYS Sunucu Tasarımı

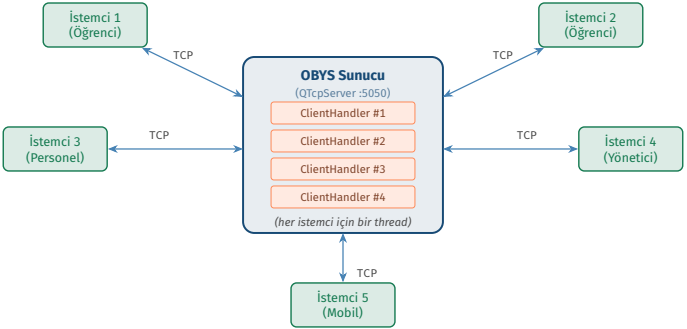
Ünite 1 – Öğrenme Hedefleri

Bu ünitede öğreneceğin

- Çok istemcili sunucunun katmanlı mimarisi
- Dinleyici, oturum, komut yönlendirici, servis, depo katmanları
- Tek-süreç, çok-süreç ve thread-pool karşılaştırması
- Bağlantı yaşam döngüsü ve durum makinesi
- Thread-per-connection deseni: avantaj/dezavantaj
- C10K problemi ve ölçek sınırları

Çok İstemcili OBYS Sunucu Topolojisi

Çoklu İstemci – Tek Sunucu Topolojisi



Sunucu her gelen bağlantı için ayrı bir ClientHandler thread'i ayırır; istemciler eşzamanlı olarak hizmet alır.

Sunucunun Beş Sorumluluk Katmanı

Katman	Sorumluluk
Dinleyici	TCP `accept`, yeni soketleri içeri alır
Oturum (Session)	İstemci başına durum: kimlik, jeton, tampon
Komut Yönlendirici	JSON `komut` alanına göre servisi seçer
Servis Katmanı	İş mantığı: doğrula, yetki kontrolü, çağrı
Depo Katmanı	Bölüm 11'in thread-safe `Depo<T>`'su

Tek sorumluluk ilkesi

Her katmanın **tek** işi vardır. Sunucu kodu bir devasa `main` fonksiyonu olarak yazılırsa bakım imkânsız hale gelir; katmanlama sayesinde hata ve test izole olur.

Eşzamanlılık Mimarisi: Üç Seçenek

Yaklaşım	Tek-süreç	Çok-süreç (fork)	Thread-pool
Eşzamanlılık	Yok / asenkron	İstemci başı süreç	Sabit thread havuzu
Bellek paylaşımı	Doğal	Yok (IPC gerek)	Doğal (mutex şart)
İzolasyon	Yok	Çok güçlü	Zayıf
Maliyet	Düşük	Yüksek (fork)	Orta
Ölçek	Tek çekirdek	Süreç sayısı	Yüzlerce/binlerce

OBYS için seçim

Eğitsel sadelik için **thread-per-connection** ile başlıyoruz; Ünite 1 Konu 3'te avantaj/dezavantajını işliyoruz. Üretim ölçeği gerekirse **thread-pool** ile epoll/IOCP'ye geçilir.

Bağlantı Yaşam Döngüsü

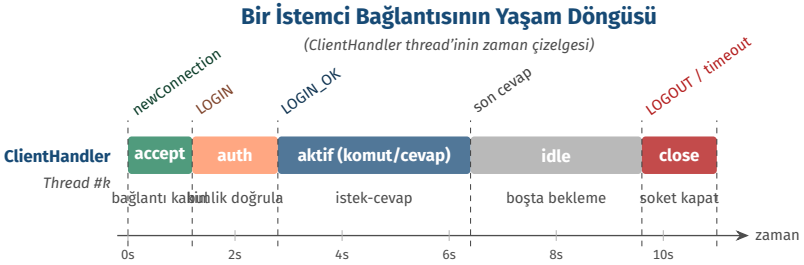
Bir bağlantının altı durumu

1. **Kabul edildi**: `accept` döndü, soket aktif
2. **Kimlik doğrulanıyor**: LOGIN bekleniyor
3. **Hazır**: Komutlar kabul ediliyor
4. **Komut işliyor**: Servise yönlendirildi
5. **Idle**: Cevap gönderildi, yeni komut bekleniyor
6. **Kapanıyor**: `disconnected` – temizlik

Durum makinesi yardımı

Her oturumun durumunu `enum` olarak tutmak; geçersiz geçişleri (örn. kimlik doğrulanmadan komut) sunucu seviyesinde **erkenden** reddetmek için kritiktir.

Thread-per-Connection: Yaşam Döngüsü



Her istemci bağlantısı; kabul, kimlik doğrulama, aktif veri alış-verişi, boşta bekleme ve kapanış evrelerinden geçer.

Thread-per-Connection: Avantajlar

Üç güçlü yan

- **Kavramsal sadelik**: Her thread tek istemciye odaklanır; akış lineer, asenkron callback labirenti yok.
- **İzolasyon**: Bir istemcinin yavaşlığı / hatası diğerlerine sirayet etmez ("bir kötü elma sepeti çürütmez").
- **Bloklayıcı I/O lüksü**: `recv` bekleyebilir; sadece o thread durur, sunucu çalışmaya devam eder.

Eğitsel kazanım

C++ `std::thread`, Qt worker, thread-safe yapılar ve oturum yönetimini bir arada görmenin en saydam yolu bu desendir.

Thread-per-Connection: Dezavantajlar ve C10K

Sınırlar

- Her thread ∼1 MB stack kullanır
- 10.000 istemci = 10 GB stack belleği
- Bağlam değiştirme (context switch) maliyeti üstel artar
- Çekirdekteki thread tablosu şişer
- İstemcilerin çoğu *idle* dururken kaynak israfı

C10K problemi

"Tek sunucuda 10.000 eş zamanlı bağlantı." Klasik thread-per-connection bu noktada çöker. Çözüm: **thread-pool + epoll/IOCP/kqueue** ya da reaktör/proaktör desenleri.

OBYS için pragmatik karar

Bir okul için 10K istemci uç durumdur. 100-500 eş zamanlı kullanıcı thread-per-connection ile rahat çalışır.

Sunucu İskeleti (Pseudo-Kod)

```
class OBYSSunucu : public QObject {
    Q_OBJECT
    QTcpServer* dinleyici_;
    QHash<QTcpSocket*, std::shared_ptr<Oturum>> oturumlar_;
    KomutYonlendirici yonlendirici_;
public slots:
    void yeniBaglanti() {
        while (dinleyici_->hasPendingConnections()) {
            QTcpSocket* s = dinleyici_->nextPendingConnection();
            auto oturum = std::make_shared<Oturum>(s);
            oturumlar_[s] = oturum;
            connect(s, &QTcpSocket::readyRead, this,
                    [this, s] { mesajIsle(s); });
            connect(s, &QTcpSocket::disconnected, this,
                    [this, s] { oturumKapat(s); });
        }
    }
};
```

Tasarım kararı

Tek event loop + her istemci için `Oturum` nesnesi. Thread'i daha sonra ekleyebiliriz; iskelet katmanlanmış.

Ünite 2: OBYS İstemci Tasarımı

Ünite 2 – Öğrenme Hedefleri

Bu ünitede öğreneceğin

- İstemci mimarisi: ağ / oturum / model / GUI katmanları
- Asenkron istemcinin altın kuralı: GUI bloklanmaz
- Bağlantı durum makinesi
- Kimlik doğrulama akışı (challenge-response taslağı)
- Komut/cevap deseni ve `request_id` korelasyonu
- Asenkron callback tablosu (`pending_requests_`)
- Yeniden bağlanma ve exponential backoff

İstemci Katmanları

Katman	Sorumluluk
Ağ Katmanı	`QTcpSocket`, çerçeveleme, byte tampon
Oturum Katmanı	Durum makinesi, jeton, kimlik bilgileri
Komut/Cevap Katmanı	JSON üret/parse, request_id eşleme
Model Katmanı	`QAbstractItemModel` – View'lere veri
GUI Katmanı	Pencereler, butonlar, sinyal/slot

Bölüm 11 mirası

Bölüm 11 "responsive UI" deseni hazır: GUI worker'a sinyal yollar, worker bitince geri sinyal döner. Bu hafta worker yerine **ServerConnection** koyuyoruz; GUI bunun farkına bile varmaz.

Asenkron İstemci: Altın Kural

GUI thread'inde `waitFor*` yasaktır

`waitForConnected`, `waitForReadyRead`, `waitForBytesWritten`: hepsi bloklayıcıdır. Ağ 2 sn sürerse Windows "Yanıt vermiyor" yazar, kullanıcı uygulamayı kapatır.

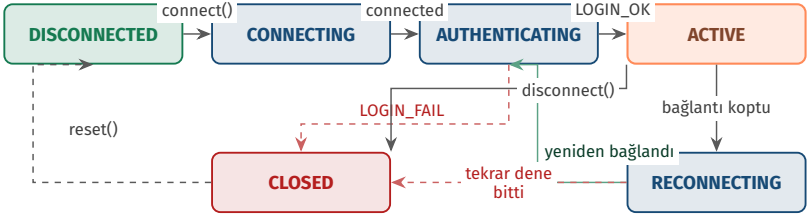
Doğru desen

- `connectToHost` – hemen döner
- `connected` sinyalini bekle
- `readyRead` ile veri al
- `disconnected` ile temizle

Felsefe

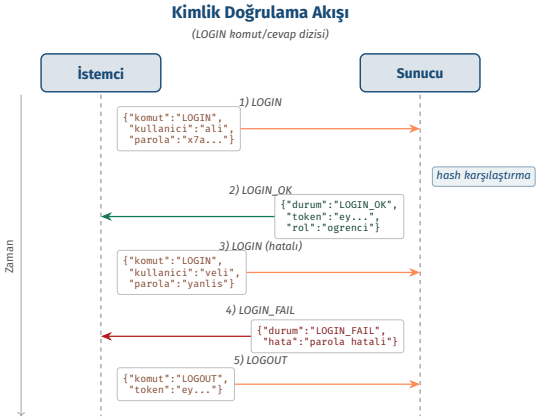
Bir saniyeyi bile bloklamayan, sinyal/slot ile haberleşen istemci **her zaman** responsive kalır.

İstemci Bağlantı Durum Makinesi



Etiket biçimi: olay / koşul. Kesikli kırmızı oklar hata yollarını, kesikli gri oklar pasif geçişleri gösterir. ACTIVE durumunda istemci komut/cevap protokolü ile sunucu ile haberleşir.

Kimlik Doğrulama Akışı



Akış: İstemci `LOGIN` gönderir; sunucu parola hash'ini saklı değerle karşılaştırır. Başarılıysa `LOGIN_OK` ve oturum `token`'ı döner; hatalıysa `LOGIN_FAIL` ile reddedilir.

Kimlik Doğrulama: Dört Adım

LOGIN dansı

1. İstemci: `LOGIN_TALEBI {kullanici}` gönderir
2. Sunucu: kullanıcının *salt*'ını ve *nonce*'unu yollar
3. İstemci: `H(salt+parola)` hesaplar; `nonce` ile ikinci hash üretip sunucuya gönderir
4. Sunucu: kendi tablosundan aynı değeri hesaplar; eşleşirse **oturum jetonu** döner

Düz parola ağa çıkmaz

Parola sadece istemci RAM'inde anlık vardır; ağda yalnızca hash dolaşır. Asıl güvenlik için bunun *üstüne* TLS gerekir (**Ünite 4**).

Komut/Cevap ve request_id

```
// Istemci -> sunucu
{
  "v": 1,
  "id": "req-42",                    // korelasyon kimligi
  "komut": "ogrenci_listele",
  "veri": { "sayfa": 0, "boyut": 50 },
  "jeton": "a3f9c2..."               // oturum jetonu
}

// Sunucu -> istemci
{
  "v": 1,
  "id": "req-42",                    // ayni id ile geri donuyor
  "durum": "ok",
  "veri": { "kayit": [ ... ] }
}
```

Niye id?

Asenkron istemci aynı anda 5 farklı komut yollayabilir; cevaplar sırasız gelebilir. `id` alanı doğru callback'e eşleşmeyi sağlar.

pending_requests_ Tablosu

```
class ServerConnection : public QObject {
    Q_OBJECT
    QTcpSocket* soket_;
    QHash<QString, std::function<void(QJsonObject)>> pending_requests_;
public:
    void komutGonder(const QString& komut,
                     const QJsonObject& veri,
                     std::function<void(QJsonObject)> cb) {
        QString id = QUuid::createUuid().toString();
        pending_requests_[id] = std::move(cb);
        QJsonObject istek{{"v",1},{"id",id},{"komut",komut},{"veri",veri}};
        gonder(istek);
    }
private slots:
    void cevapGeldi(QJsonObject cevap) {
        QString id = cevap["id"].toString();
        if (auto it = pending_requests_.find(id); it != pending_requests_.end()) {
            it.value()(cevap);
            pending_requests_.erase(it);
        }
    }
};
```

Yeniden Bağlanma: Exponential Backoff

Niye düz aralıkla deneme yetmez?

İstemci kopunca her saniye yeniden bağlanmaya çalışırsa, sunucu gerçekten geçici olarak yıkılmışsa milyonlarca istek altında ezilir. Üstüne 1000 istemci aynı şeyi yaparsa felaket büyür.

Strateji

Bekleme süresi her başarısız denemede **ikiye katlanır**: `1s`, `2s`, `4s`, `8s`, `16s`, `32s`, `60s` (üst sınır). Üstüne küçük rastgele bir miktar eklenir (*jitter*) ki tüm istemciler aynı anda saldırmasın.

Üst sınır şart

Sonsuz büyüyen backoff istemciyi pratikte ölü hale getirir; 60 sn gibi makul bir tavan uygula ve kullanıcıya durumu göster.

Ünite 3: Veri Senkronizasyonu

Ünite 3 – Öğrenme Hedefleri

Bu ünitede öğreneceğin

- Sunucu tarafında Bölüm 11 `Depo<T>` entegrasyonu
- `shared_mutex` ile okuyucu/yazar kalıbı
- Eş zamanlı yazma çakışması: last-write-wins vs versiyon
- Audit log: amaç, yapı, append-only kuralı
- Asenkron log yazımı (üretici-tüketici)
- Checkpoint stratejisi ve atomik dosya yazımı

Bölüm 11'i Hatırlayalım

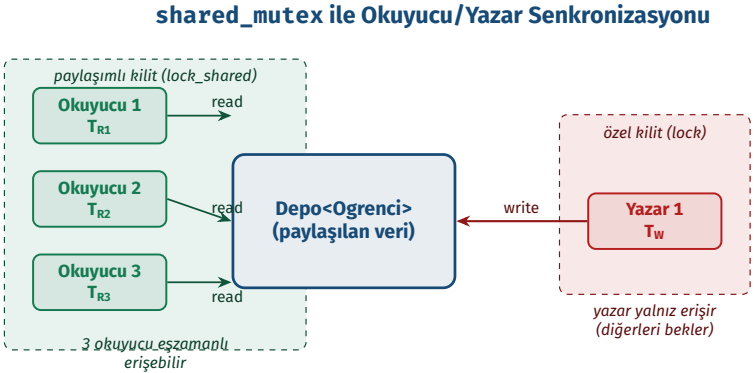
Hazır thread-safe yapılarımız

- `ThreadSafeQueue<T>` – üretici-tüketici için
- `ConcurrentMap<K,V>` – shard'lı, çakışma az
- `shared_mutex` – okuyucu/yazar kilidi
- `stop_token`, `jthread` – C++20

Bu hafta sunucu çekirdeğine bunları yerleştiriyoruz

Sunucumuzun `Depo<Ogrenci>` ve `Depo<NotKaydi>` nesneleri farklı thread'lerden eş zamanlı çağrılır. Bu çağrıları güvenli kılan kilit stratejisi seçimi **performans/güvenlik** dengesini belirler.

shared_mutex: Okuyucu/Yazar



Kural: `shared_mutex`, çok sayıda okuyucunun aynı anda erişimine izin verir; ancak yazar gelince diğer tüm thread'ler beklemeye geçer ve yazar tek başına çalışır.

shared_mutex Kullanımı

```
class NotDepo {
    mutable std::shared_mutex m_;
    std::map<int, NotKaydi> kayitlar_;
public:
    std::optional<NotKaydi> bul(int no) const {
        std::shared_lock kilit(m_);          // OKUYUCU: paylasimli
        auto it = kayitlar_.find(no);
        return it != kayitlar_.end()
            ? std::optional{it->second} : std::nullopt;
    }
    void guncelle(int no, NotKaydi yeni) {
        std::unique_lock kilit(m_);          // YAZICI: ozel
        kayitlar_[no] = std::move(yeni);
    }
};
```

Performans kazancı

28 öğretmen aynı anda `bul` çağırırsa hepsi paralel okur. Standart `mutex` ile bunlar sıraya girerdi.

Eş Zamanlı Yazma Çakışması

Senaryo

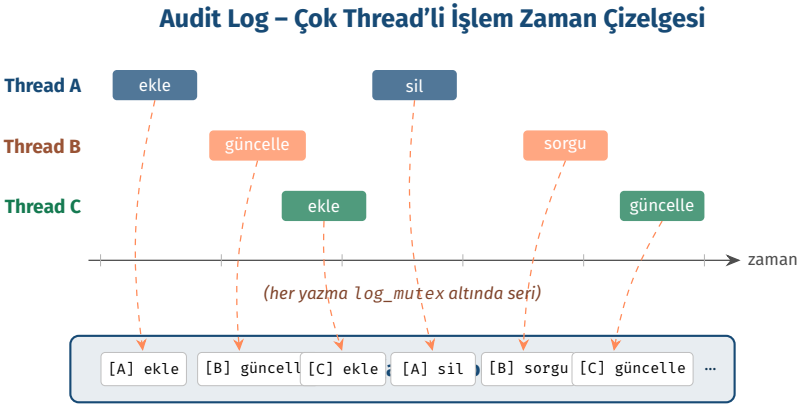
İki öğretmen aynı öğrencinin notunu farklı değerlerle aynı anda günceller. Sunucu hangisini kabul etmeli? **Kaybedilen güncelleme** problemi.

Strateji	Avantaj	Dezavantaj
Last-write-wins	Basit, hızlı	Sessizce veri kaybı
Versiyon (optimistik)	Çakışma görünür, güvenli	İstemci tekrar denemeli
Pessimistik kilit	Çakışma yok	Kullanıcı bekler, deadlock riski

OBYS tercihi

Not güncellemede **versiyon**. `kayit_v=7` bekleyen istek sunucuda v=8 görürse `409 Conflict` döner; istemci farkı kullanıcıya gösterir.

Audit Log Zaman Çizelgesi



Audit log: Üç thread bağımsız işlem yaparken her olay tek bir `audit.log` dosyasına sıralı olarak eklenir; mutex sayesinde girdiler birbirine karışmaz.

Audit Log: Niye ve Nasıl?

İki amaç

- **Denetlenebilirlik**: Kim, ne zaman, hangi veriyi değiştirdi? KVKK ve denetim için yasal şart.
- **Felaket kurtarma**: Çöküş sonrası son checkpoint + log oynatımı ile son tutarlı duruma erişim (write-ahead logging mantığı).

Append-only kuralı

Audit log dosyası asla yerinde değiştirilmez; sadece sonuna eklenir. İşletim sistemi düzeyinde `O_APPEND` bayrağı ile açılır. Bu sayede yazılan kayıt geriye dönük tahrif edilemez.

Asla loglama

Düz parola, oturum jetonu, kart numarası, kişisel sağlık verisi. Loglar saldırgan için altın madenidir.

Asenkron Log Yazımı (Üretici-Tüketici)

Tasarım

İstemci thread'leri `LogKaydi`'nı **thread-safe kuyruğa** bırakır ve hemen işine döner. Tek bir *logger thread* kuyruktan alıp dosyaya yazar.

Avantaj

- İstemci thread'i disk I/O için beklemez (latency düşer)
- Disk yazımı sıralı (kafa hareketi optimum)
- Tek dosya tutamacı → bayrak yarışı yok
- Bölüm 10'daki üretici-tüketici deseninin doğal uygulaması

Risk

Sunucu çökerse kuyrukta bekleyen kayıtlar kaybolur. Kritik kayıtlar için `fsync` ile senkron flush gerekir.

Checkpoint ve Atomik Dosya Yazımı

Checkpoint

Sunucunun belirli aralıklarla (örn. 5 dk) tüm depo durumunu diske *snapshot* olarak yazmasıdır. Açılışta checkpoint yüklenir, ardından **checkpoint sonrası log oynatılır**.

Atomik dosya yazımı (write-then-rename)

1. `depo.dat.tmp` dosyasına yaz
2. `fsync` ile diske bas
3. `rename(depo.dat.tmp, depo.dat)` – atomiktir

Niye `rename`?

Doğrudan üzerine yazma yarısında çöküşte dosyayı bozar. POSIX `rename` aynı dosya sisteminde atomiktir: ya eski ya yeni, ortada kalmış hibrit yok.

Ünite 4: Güvenlik Temelleri

Ünite 4 – Öğrenme Hedefleri

Bu ünitede öğreneceğin

- Tehdit modeli: hangi saldırgana karşı koruyoruz?
- Parola hash pipeline: `QCryptographicHash`
- Salt + slow hash (Argon2 / bcrypt) kavramları
- TLS el sıkışma (handshake) ve dört güvenlik garantisi
- Girdi doğrulama kategorileri
- Yetkilendirme matrisi (rol/izin)
- En az ayrıcalık (principle of least privilege)

Tehdit Modeli

Hangi saldırgana karşı?

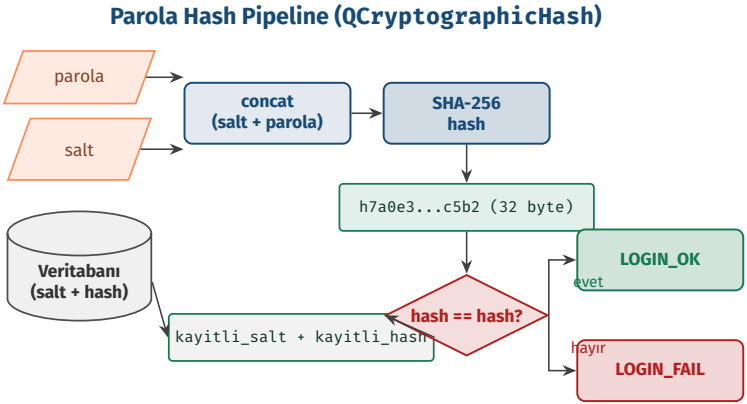
Güvenlikte ilk soru hep: *kimden* korunuyorsun?

Tehdit Seviyesi	**Yapabildiği**
Pasif dinleyici	Ağ trafiğini görür (TLS koruması)
Aktif saldırgan (MITM)	Trafiği değiştirir (TLS + sertifika)
Veritabanı sızdırıcı	Disk dosyasını alır (salt + slow hash)
Sunucuyu ele geçiren	RAM'e erişir (savunma çok zor)

Katmanlı savunma

Tek bir önlem yetmez. TLS **ve** salt + hash **ve** girdi doğrulama **ve** yetkilendirme. Defense-in-depth.

Parola Hash Pipeline



Pipeline: Kullanıcının parolasına veritabanındaki salt eklenir, SHA-256 ile hash'lenir; üretilen değer kayıtlı hash ile sabit-zamanlı (constant-time) karşılaştırılır. Eşleşirse oturum açılır.

QCryptographicHash Kullanımı

```
QByteArray hashUret(const QString& parola, const QByteArray& salt) {
    QCryptographicHash h(QCryptographicHash::Sha256);
    h.addData(salt);
    h.addData(parola.toUtf8());
    return h.result();
}
```

Bu kadar yetmez

SHA-256 **hızlıdır**: saniyede milyarlarca deneme yapılabilir. GPU'lu saldırgan birkaç gün içinde yaygın parolaları kırar.

Doğru tercih: yavaş hash

- **bcrypt**: ayarlanabilir maliyet faktörü
- **Argon2**: bellek-zor (GPU'ya direnç)
- **PBKDF2**: çok-tur SHA, FIPS uyumlu

Maliyet: tek doğrulama 100-500 ms (kullanıcı fark etmez, saldırgan *çok* fark eder).

Salt: Niye Vazgeçilmez?

Salt'sız hash

Aynı parolayı kullanan iki kullanıcı aynı hash'e sahip olur. **Rainbow table** saldırganı her popüler parolanın hash'ini önceden hesaplar; tek tablo bakışında parolayı bulur.

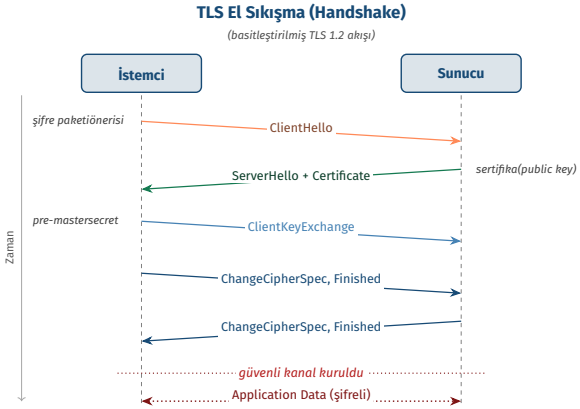
Salt ile

Her kullanıcıya **benzersiz rastgele** bayt dizisi (16 B). Aynı parola, farklı salt → farklı hash. Saldırgan rainbow table kullanamaz; her kullanıcı için sıfırdan hesap gerekir.

Salt nerede saklanır?

Veritabanında, parola hash'inin yanında. Salt **gizli değildir**; ama her kullanıcıya özeldir. Saldırgan saltı bilse de yine her kullanıcı için ayrı brute-force çalıştırmak zorunda.

TLS El Sıkışma



Akış: İstemci destekli şifre paketlerini ClientHello ile bildirir; sunucu sertifika paylaşır. Anahtar değişiminden sonra her iki taraf da `Finished` ile el sıkışmayı bitirir ve veri AES gibi simetrik bir şifre ile aktarılır.

TLS'in Dört Garantisi

Garanti	Anlamı
Gizlilik	Üçüncü taraf trafiği okuyamaz (simetrik şifreleme)
Bütünlük	Trafik değiştirilirse alıcı fark eder (HMAC)
Kimlik kanıtı	Sunucu sertifika ile kim olduğunu kanıtlar
İleri gizlilik	Anahtar geleceğe sızsa eski trafik açılmaz

Qt'de TLS

`QSslSocket`, `QTcpSocket`'in tüm sinyallerini destekler; ek olarak `encrypted`, `sslErrors`. `QTcpServer` yerine `QSslServer` (Qt 6.4+) ile dinleme yapılır. Sertifika ve özel anahtar yükleme şarttır.

Self-signed sertifika

Geliştirmede yeterli; üretimde Let's Encrypt veya kurumsal CA.

Girdi Doğrulama: Niye ve Nasıl?

Sunucu için altın kural

Hiçbir girdiye güvenme. Şifreli tünelin içinden gelse bile içerik dürüst olmayabilir; istemci kötü niyetli, hatalı veya sadece eski sürüm olabilir.

Üç geçişli süzgeç

1. **Şema**: JSON geçerli mi? Beklenen alanlar, tipler, boyutlar?
2. **İş mantığı**: Not 0-100 arasında mı? Tarih makul mü? Yabancı anahtar var mı?
3. **Yetkilendirme**: Bu kullanıcı bu işi bu kayıtta yapabilir mi?

Sıralama önemli

Ucuzdan pahalıya: önce şema (anında reddet), sonra iş mantığı (veri okur), en sonda yetki (rol+kayıt sorgusu).

Girdi Doğrulama Kategorileri

Kategori	Örnek
Tip	String mi? Int mi? Bool mu?
Boyut/uzunluk	Ad $\leq$ 100 karakter; payload $\leq$ 16 MB
Aralık	Not 0-100; sayfa boyutu 1-200
Format/desen	E-posta regex; tarih ISO-8601
Beyaz liste	Bölüm kodu sabit listede mi?
Referans bütünlüğü	ogr_no var mı? Ders kodu kayıtlı mı?

Kara liste yerine beyaz liste

"Şu karakterler yasak" yaklaşımı her zaman geride kalır. "Sadece şu karakterler izinli" yaklaşımı tutarlıdır.

Yetkilendirme Matrisi (Rol / İzin)

Rol \ İzin	ogr_oku	ogr_yaz	not_yaz	rapor
Öğrenci	sadece kendi	–	–	–
Öğretmen	ders öğrencileri	–	ders notları	ders raporu
Sekreter	tümü	ekle/güncelle	–	–
Admin	tümü	tümü	tümü	tümü

Pratik uygulama

Her komut işleyicisinde: `if (!yetkili(kullanici, "not_yaz", ogr_no)) reddet();` Tek satır, ama tutarlı uygulanması şart.

Veri seviyesinde yetki

"not_yaz" iznine sahip öğretmen *her* öğrencinin notunu değil, sadece *kendi dersindeki* öğrencinin notunu değiştirebilir. Bu **kayıt-bazlı** yetki sorgusu gerektirir.

En Az Ayrıcalık Prensibi

Principle of Least Privilege (PoLP)

Her bileşen / kullanıcı sadece **görevi için zorunlu olan** yetkiye sahip olmalıdır. Bir gram fazlası saldırı yüzeyini büyütür.

Sunucu tarafında uygulama

- Sunucu süreci **root olarak çalışmaz**; kendi kullanıcısı vardır
- Veritabanı bağlantısı sadece OBYS şemasına erişir
- Audit log dosyası ayrı kullanıcının yazma iznine sahiptir
- Bağlantı port'u 1024 üstüdür (root gerekmesin)

Tarihsel ders

Production sunucuların büyük güvenlik felaketlerinin çoğu PoLP ihlalinden gelir: gerek yokken root, gerek yokken tüm DB erişimi, gerek yokken sınırsız dosya erişimi.

Bu Hafta Neyi Başardık?

Sunucu (Ünite 1)

- Katmanlı mimari: dinleyici, oturum, yönlendirici, servis, depo
- Thread-per-connection deseni ve C10K sınırı

İstemci (Ünite 2)

- Asenkron `ServerConnection`, durum makinesi, kimlik doğrulama
- `request_id` korelasyonu, exponential backoff

Veri (Ünite 3)

- `shared_mutex`, versiyon ile çakışma yönetimi
- Audit log + checkpoint + atomik dosya yazımı

Güvenlik (Ünite 4)

OBYS'nin On Üç Haftalık Yolculuğu

Bölüm 1-13: Tek satır özeti

- B1-3: STL ile `Depo<T>`, modern C++
- B4-8: Qt GUI, Model/View, ileri bileşenler
- B9-10: Thread, mutex, async, QThread
- B11: Thread-safe yapılar, responsive UI
- B12: Ağ temelleri, soket, JSON protokol taslağı
- **B13: Tam istemci/sunucu OBYS, eş zamanlı veri, güvenlik**

Toplam birikim

Tek makine → GUI → Çok kanallı → Ağa açık → **Güvenli, ölçeklenebilir dağıtık OBYS**.

14. Hafta Önizleme: Final, Test, Dağıtım

Bölüm 14: Üretime hazırlık

- Birim ve entegrasyon testleri (Qt Test, GoogleTest)
- CI/CD: derleme, test, paketleme otomasyonu
- Paketleme: Linux `.deb`, Windows kurulum, macOS `.app`
- Loglama, izleme, telemetri
- Sürüm numaralama ve dağıtım kanalları
- Final proje teslimi

Kapanış mesajı

Profesyonel yazılım yalnızca çalışan koddan ibaret değildir; **ölçülebilir, test edilebilir, dağıtılabilir, güvenli ve sürdürülebilir** olmalıdır. 14. hafta bu zinciri tamamlar.

Sorularınız?

Bölüm 13 – OBYS Client/Server Mimarisi
Yazılım Geliştirme II